Лабораторная работа № 20: Исследование методов управления маршрутами с использованием объекта HTTP Request.

Цель работы: Сформировать умение анализировать методы управления маршрутами с использованием объекта HTTP Request.

Задание 1 (Базовый уровень)

Создание простого HTTP-сервера на чистом модуле `http`

Описание:
Необходимо создать базовый HTTP-сервер без использования фреймворков (Koa, Express), используя только модуль `http`. Сервер должен обрабатывать запросы и отдавать ответы.

Как это должно работать:

1. Сервер слушает порт 3000
2. При запросе `GET /` возвращается HTML-страница с информацией о группе ББМО-01-23
3. При запросе `GET /about` возвращается информация о студенте
4. При запросе к неизвестному пути — 404 с HTML-страницей
5. Логируются все входящие запросы (метод, URL, время)

Пример работы:

```
text
```

```
$ node http-server.js
Сервер запущен на порту 3000

# В браузере: http://localhost:3000/
=== Лабораторная работа №20 ===
Группа: ББМО-01-23
Метод: GET
URL: /

# В браузере: http://localhost:3000/about
=== О студенте ===
Группа: ББМО-01-23
Студент: [Ваше ФИО]

# В браузере: http://localhost:3000/unknown
=== 404 Not Found ===
Путь /unknown не найден
```

```
# В консоли сервера:
[2024-01-15 14:30:25] GET / 200
[2024-01-15 14:30:28] GET /about 200
[2024-01-15 14:30:30] GET /unknown 404
```

Что нужно сделать для отчёта:

- Скриншот запуска сервера
- Скриншот запроса `/`
- Скриншот запроса `/about`
- Скриншот запроса `/unknown` (404)
- Скриншот логов сервера

Задание 2 (Средний уровень)

Работа с HTTP-заголовками: чтение и установка

Описание:
Изучить методы работы с HTTP-заголовками: чтение входящих заголовков и установка исходящих.

Как это должно работать:

1. Чтение заголовков запроса:
    - Все заголовки доступны через `request.headers`
    - Все ключи автоматически в нижнем регистре
    - Специальные заголовки: `user-agent`, `accept`, `host`, `content-type`
2. Эндпоинт `GET /headers`:
    - Возвращает JSON со всеми заголовками запроса
    - Дополнительно: `httpVersion`, `method`, `url`
3. Установка заголовков ответа:
    - `Content-Type: application/json` для JSON-ответов
    - `Content-Type: text/html` для HTML-ответов
    - `X-Powered-By: Node.js`
    - `X-Group: ББМО-01-23`
    - `Cache-Control: no-cache`
4. Методы работы с заголовками:
    - `response.setHeader(name, value)` — установить
    - `response.getHeader(name)` — проверить
    - `response.hasHeader(name)` — существует ли
    - `response.removeHeader(name)` — удалить до отправки
5. Демонстрация:

- Эндпоинт `GET /headers/set` устанавливает кастомные заголовки
- Эндпоинт `GET /headers/check` проверяет наличие заголовка

Пример работы:

```
text

# Запрос: GET /headers
$ curl http://localhost:3000/headers
{
  "headers": {
    "host": "localhost:3000",
    "user-agent": "curl/7.68.0",
    "accept": "*/*"
  },
  "method": "GET",
  "url": "/headers",
  "httpVersion": "1.1"
}

# Запрос: GET /headers/set
$ curl -I http://localhost:3000/headers/set
HTTP/1.1 200 OK
Content-Type: text/html
X-Powered-By: Node.js
X-Group: ББМО-01-23
Cache-Control: no-cache

# Запрос: GET /headers/check
$ curl http://localhost:3000/headers/check
{
  "hasContentType": true,
  "hasXGroup": true,
  "hasUnknown": false

}
```

Что нужно сделать для отчёта:

- Скриншот `GET /headers` (все заголовки)
- Скриншот `GET /headers` с кастомным User-Agent
- Скриншот `GET /headers/set` (установленные заголовки)
- Скриншот `GET /headers/check`
- Скриншот проверки через `curl -I`

Задание 3 (Продвинутый уровень)

Чтение тела запроса: работа с POST-данными

Описание:
Изучить методы чтения тела запроса. Тело — это `Readable Stream`, данные приходят кусками (chunks), которые нужно собирать вручную.

Как это должно работать:

1. Чтение тела через события `data` и `end`:
    - `request.on('data', chunk => ...)` — накопление данных
    - `request.on('end', () => ...)` — обработка после получения всего тела
    - `request.on('error', err => ...)` — обработка ошибок
2. Обработка разных Content-Type:
    - `application/json` — парсинг JSON
    - `application/x-www-form-urlencoded` — парсинг form-data
    - `text/plain` — как есть
    - `application/octet-stream` — бинарные данные
3. Эндпоинт `POST /echo`:
    - Принимает любое тело
    - Возвращает его обратно с указанием Content-Type
    - Пример: `{"message": "Hello"}` → возвращает то же самое
4. Эндпоинт `POST /form`:
    - Принимает form-data
    - Парсит в объект
    - Возвращает распарсенные данные
5. Ограничение размера тела:
    - Максимальный размер — 1 МБ
    - При превышении — 413 Payload Too Large
    - Демонстрация защиты от переполнения
6. Эхо-сервер для группы ББМО-01-23:
    - При получении JSON добавляет поле `"group": "ББМО-01-23"`
    - Логирует размер тела

Пример работы:

```
text

# POST /echo с JSON
$ curl -X POST http://localhost:3000/echo \
  -H "Content-Type: application/json" \
  -d '{"message": "Hello"}'
{
  "message": "Hello",
  "group": "ББМО-01-23"
}

# POST /echo с text/plain
```

```
$ curl -X POST http://localhost:3000/echo \
  -H "Content-Type: text/plain" \
  -d 'Hello, world!'
Hello, world!

# POST /form
$ curl -X POST http://localhost:3000/form \
  -d 'name=Иван&group=ББМО-01-23'
{
  "name": "Иван",
  "group": "ББМО-01-23"
}

# Превышение размера
$ curl -X POST http://localhost:3000/echo \
  -H "Content-Type: application/json" \
  -d @large-file.json

413 Payload Too Large
```

Что нужно сделать для отчёта:

- Скриншот `POST /echo` с JSON
- Скриншот `POST /echo` с text/plain
- Скриншот `POST /form`
- Скриншот ошибки 413 при превышении размера
- Скриншот логов с размером тела

Задание 4 (Сложный уровень)

Маршрутизация на основе `request.url` и `request.method`

Описание:
Реализовать полноценную маршрутизацию без фреймворков, используя только `request.url` и `request.method`. Поддержать параметры пути, query-параметры и REST API.

Как это должно работать:

1. Парсинг URL:
    - Использовать модуль `url` (`url.parse()` или `new URL()`)
    - Разделение пути и query-параметров
    - Декодирование URL-encoded значений
2. Параметры пути:
    - `/users/:id` — параметр `id`
    - `/users/:userId/posts/:postId` — несколько параметров

    - Регулярные выражения для匹配 маршрутов
3. Query-параметры:
    - `/search?q=hello&limit=10` — параметры `q` и `limit`
    - Парсинг в объект
    - Значения по умолчанию
4. REST API для студентов группы ББМО-01-23:
    - `GET /api/students` — все студенты
    - `GET /api/students?group=ББМО-01-23` — фильтрация
    - `GET /api/students/:id` — один студент
    - `POST /api/students` — создание
    - `PUT /api/students/:id` — обновление
    - `DELETE /api/students/:id` — удаление
5. Маршрутизация:
    - Таблица маршрутов: `{ method, pattern, handler }`
    - Поиск подходящего маршрута
    - 404 — если не найден
    - 405 — если метод не поддерживается
6. Логирование:
    - Каждый запрос логируется с методом, URL, статусом
    - Для группы ББМО-01-23 — добавить префикс

Пример работы:

```
text
```

```
# GET /api/students
$ curl http://localhost:3000/api/students
[
  {"id": 1, "name": "Иван", "group": "ББМО-01-23"},
  {"id": 2, "name": "Мария", "group": "ББМО-01-23"}
]

# GET /api/students?group=ББМО-01-23
$ curl "http://localhost:3000/api/students?group=ББМО-01-23"
[{"id": 1, "name": "Иван", "group": "ББМО-01-23"}]

# GET /api/students/1
$ curl http://localhost:3000/api/students/1
{"id": 1, "name": "Иван", "group": "ББМО-01-23"}

# POST /api/students
$ curl -X POST http://localhost:3000/api/students \
  -H "Content-Type: application/json" \
  -d '{"name": "Петр", "group": "ББМО-01-23"}'
{"id": 3, "name": "Петр", "group": "ББМО-01-23"}

# PUT /api/students/1
$ curl -X PUT http://localhost:3000/api/students/1 \
  -H "Content-Type: application/json" \
```

```
  -d '{"name": "Иван Иванов"}'
{"id": 1, "name": "Иван Иванов", "group": "ББМО-01-23"}

# DELETE /api/students/1
$ curl -X DELETE http://localhost:3000/api/students/1
{"deleted": true, "id": 1}

# 405 Method Not Allowed
$ curl -X PATCH http://localhost:3000/api/students/1
405 Method Not Allowed

# Логи
[2024-01-15 14:30:25] [ББМО-01-23] GET /api/students 200
[2024-01-15 14:30:28] [ББМО-01-23] POST /api/students 201
[2024-01-15 14:30:30] [ББМО-01-23] PATCH /api/students/1 405
```

Что нужно сделать для отчёта:

- Скриншот `GET /api/students`
- Скриншот `GET /api/students?group=ББМО-01-23`
- Скриншот `GET /api/students/:id`
- Скриншот `POST /api/students`
- Скриншот `PUT /api/students/:id`
- Скриншот `DELETE /api/students/:id`
- Скриншот 405 при неподдерживаемом методе
- Скриншот логов

Задание 5 (Экспертный уровень)

Полноценный HTTP-сервер с middleware, статикой, кэшированием и HTTPS

Описание:
Создать полнофункциональный HTTP-сервер на чистом модуле `http` с поддержкой middleware, отдачи статических файлов, кэширования, HTTPS и стриминга.

Как это должно работать:

1. Middleware-система:
    - Массив функций `middleware[]`
    - Каждая функция получает `(req, res, next)`
    - `next()` передаёт управление следующему middleware
    - Обработка ошибок в middleware
2. Статические файлы:
    - `GET /static/*` — отдача файлов из папки `public/`

    - Определение MIME-типа по расширению
    - Поддержка `If-Modified-Since` и `ETag`
    - 304 Not Modified при отсутствии изменений
    - Защита от path traversal (`../`)
3. Кэширование:
    - In-memory кэш для GET-запросов
    - Время жизни — 60 секунд
    - Заголовок `Cache-Control: max-age=60`
    - Инвалидация при изменении данных
4. HTTPS:
    - Генерация самоподписанного сертификата
    - Создание HTTPS-сервера через модуль `https`
    - Сравнение HTTP и HTTPS
5. Стриминг:
    - `GET /stream` — стриминг большого файла
    - Использование `fs.createReadStream()`
    - `pipe()` для передачи в response
    - Прогресс загрузки
6. Логирование и метрики:
    - Лог всех запросов в `access.log`
    - Счётчики: всего запросов, по методам, по статусам
    - `GET /metrics` — метрики сервера
7. Обработка ошибок:
    - Слушатель `error` на сервере
    - Слушатель `clientError` для некорректных запросов
    - Корректное завершение по `SIGINT`
8. Для группы ББМО-01-23:
    - Все ответы содержат заголовок `X-Group: ББМО-01-23`
    - Метрики содержат информацию о группе

Пример работы:

```
text

$ node http-advanced.js
[INFO] HTTP-сервер запущен на порту 3000
[INFO] HTTPS-сервер запущен на порту 3443
[INFO] Группа: ББМО-01-23

# Статика
$ curl http://localhost:3000/static/index.html
<!DOCTYPE html>...

# Кэширование
$ curl -I http://localhost:3000/static/style.css
HTTP/1.1 200 OK
Cache-Control: max-age=60
```

```
ETag: "abc123"

$ curl -I http://localhost:3000/static/style.css \
  -H "If-None-Match: \"abc123\""
HTTP/1.1 304 Not Modified

# Path traversal защита
$ curl http://localhost:3000/static/../secret.txt
403 Forbidden

# Стриминг
$ curl http://localhost:3000/stream > big-file.zip
  % Total    % Received % Xferd  Average Speed
100  100M  100  100M    0     0  50.2M      0  0:00:02

# HTTPS
$ curl -k https://localhost:3443/
=== HTTPS работает! ===
Группа: ББМО-01-23

# Метрики
$ curl http://localhost:3000/metrics
{
  "group": "ББМО-01-23",
  "total": 152,
  "byMethod": { "GET": 120, "POST": 30, "DELETE": 2 },
  "byStatus": { "200": 140, "404": 10, "500": 2 },
  "uptime": 3600
}

# Логи
[2024-01-15 14:30:25] [ББМО-01-23] GET /static/index.html 200
[2024-01-15 14:30:28] [ББМО-01-23] GET /static/style.css 304
[2024-01-15 14:30:30] [ББМО-01-23] GET /static/../secret.txt 403

[2024-01-15 14:30:35] [ББМО-01-23] GET /stream 200
```

Что нужно сделать для отчёта:

- Скриншот структуры проекта
- Скриншот работы middleware
- Скриншот отдачи статики
- Скриншот кэширования (200 → 304)
- Скриншот защиты от path traversal
- Скриншот HTTPS-запроса
- Скриншот стриминга
- Скриншот метрик
- Скриншот логов
- Скриншот корректного завершения

Фиксация изменений

Добавить файлы в Git
Создать коммит
Отправить в удалённый репозиторий
Создать Pull Request